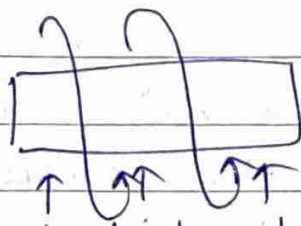


Week-11 Speech Tech

L1 Transformer - Introduction

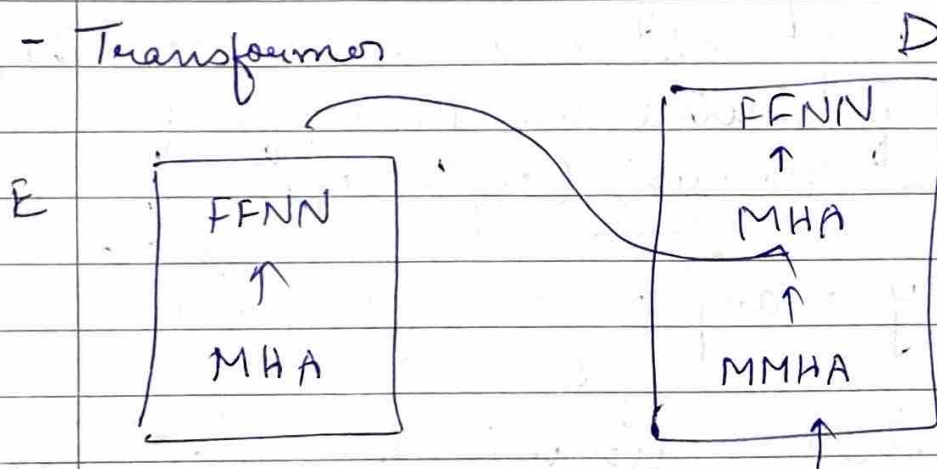
- so for RNNs & LSTMs, the process was 1-by-1.



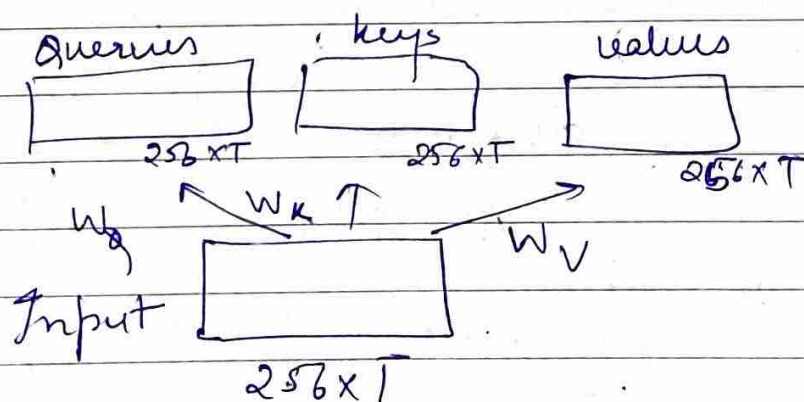
they are in sequential nature.

So we had to parallelize it. So, we used CNNs with multiple filters to capture all kinds of information.

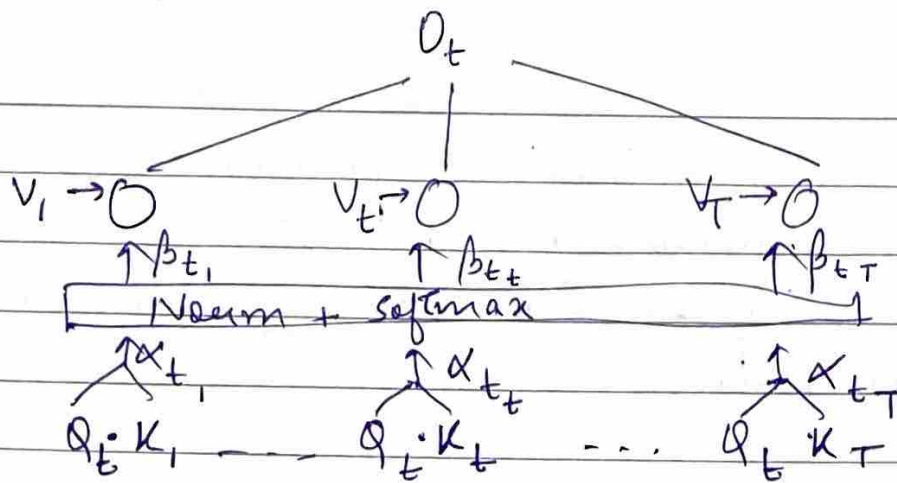
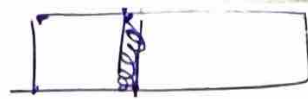
- Transformer



- self attenⁿ ensures that the distance btw. words (or acoustic features) are not important. All are treated equally.



Output



- instead of what happens in word2vec, we learn this embeddings matrix in self-attenⁿ
- BERT uses only encoder.

L2 Transformer- Self Attenⁿ

- why we came to transformers? to run everything in a parallel, sequential order.
- they look for the cosine similarity for each vectors (inner products) to see how much words are similar to each other
- let's pick 3rd vector $\rightarrow$ that becomes my query vector, now for the 3rd vector find the inner product with all the vector including itself (these are the keys)
- now as per the model we can do the computations parallelly.
- currently, the thing was even if we Δ the order of the words completely, we would get the exact same.

- _/_/_
- further, sinusoidal position embeddings were brought like a vector to the sentence embedding - to make sure word order makes sense.

L3 Transformers - Training

- In encoder, we have MHA (is the self-attenⁿ). In convoluⁿ, we capture local context. In attenⁿ, we take the entire context.
- $\therefore$, we use multiple heads to capture different types of context.
- In decoder, is sequential, we have masked MHA.
- Enc-dec., the entire training is fully parallelizable.
- During inference, encoder is parallelizable. But, the decoder is sequential.
- The attenⁿ is not affected by the distance b/w. the words.

L4 Transformers for Speech

ASR. Input audio features at encoder $\rightarrow$ Transformer $\rightarrow$ Convers. text in same lang. at output of decoder.

- ASR sys. — E2E Transformers
 Input — acoustic vectors
 Output — seq. of characters or BPEs
 Dictionary — Not needed
 & LM

This can handle new technical terms as well. They have outperformed RNN-based encoder-decoder models in ASR.

- NLTM Project

Audio Eng. Input



ASR



Text Output
in English



MT



Text Input
in Hindi

Audio output in
↑ Hindi

TTS



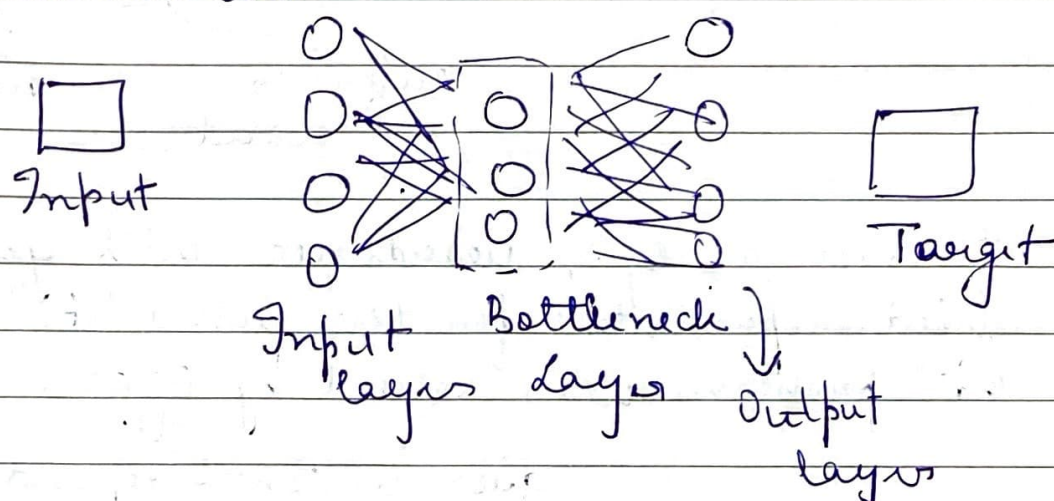
- Eg. ITM's ASR demo on sample NPTEL lectures. There might be some mistakes. But it is still quite good.

- we have now discussed methods that had supervision or parallel corpora.
- for transformers, or atten"-based enc-dec etc. parallel corpora is necessary → we introduced positional encodings.
- MT - both source text & corresponding translated target text is necessary.
- ASR - both audio & corresponding transcrip" is necessary.

L5

Intro to self-supervised learning

- supervised $\rightarrow$ needs audio & corresponding Text
Input $\rightarrow$ Model $\rightarrow$ Targets
- data is expensive, but raw data is easy to get. Motivation — How to make use of easily & largely available audio only data?
- autoencoders



L6

BERT (Bidirectional Encoder Representations from Transformer)

- purpose — to harvest benefits from text-only data.
- arch. — transformer encoder.
- tasks — masked predicⁿ & classificaⁿ.
- word2vec — self-supervised learning
 - $\hookrightarrow$ fixed representaⁿ
 - $\hookrightarrow$ inputs & outputs are one-hot vectors

using PCA

$$\begin{pmatrix} \text{the} \\ 500 \times 1 \end{pmatrix} = W \begin{pmatrix} 1 \\ \text{the} \\ 500 \times 1000 \end{pmatrix} \begin{pmatrix} 100,000 \times 1 \end{pmatrix} \rightarrow \text{embedding matrix}$$

- for all the words, embedding matrix is once calculated & gets fixed, irrespective of the context. (eg. cute cat see dirty cat) it doesn't $\downarrow$

Δ with the context -

- $\rightarrow$ 1 word - 1 embedding
- eg. animal didn't cross road, as it was tired. words.

refers to road

refers to animal

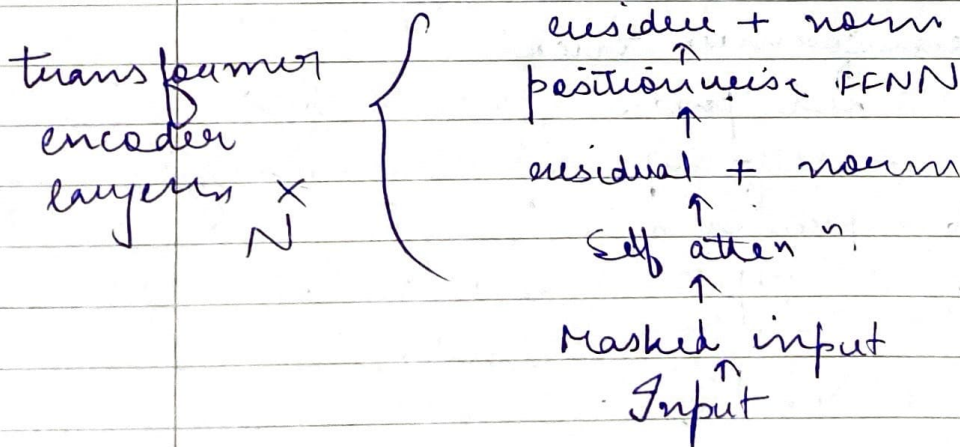
In such a eg., wordsvec will give the fixed representation for the word "it."

- This problem was solved by BERT.

$\downarrow \downarrow$
gives context-dependent word embeddings.

- So, the transformer's self-attenⁿ was calculating the dependence for each of the words using the idea of dot-product

- BERT



- few words are masked, which will get predicted at the output layer.
- it makes things easier, even with small amt. of data.
- Total Loss = NSP Loss + Masked Token Predictⁿ Loss

T/F whether the following statement is the correct adjoining sentence or not.

check whether the masked tokens have been incorrectly unmasked or not.

- BERT is trained on pairs of sentences. Each word is broken down into word-pieces & tokens. Special tokens - CLS, SEP & MASK. 50% of the time, it is the actual sequence & 50% it's not.
- The task of BERT is to reduce the combined loss, which is the sum of the NSP & MLM loss.

next sentence predictⁿ.

masked language modelling

- We use the power of non-linearity in the neural networks.